Software Design Document (Assignment 2) - Part 2: Scoring Engine Extension

Project Name: Fantasy Tactics League
Version: 2.0 (Extension Phase)

1. Project Information & Task Matrix

1.1 Team Members

- Muhammet Emir Dizdar (211401028)
- Doruk Dolaşık (211401030)
- Serhan Tezcan (211401024)

1.2 Updated Task Matrix (Part 2)

Task	Responsible Member(s)
System Overview & Extension Context	Serhan Tezcan
Scoring Engine Logic & Rationale	Muhammet Emir Dizdar
Design Patterns: Strategy & Decorator	Doruk Dolaşık
Behavioral UML (Sequence & Activity)	Muhammet Emir Dizdar
Data Flow & Object Mapping	Serhan Tezcan
Final Review & Quality Assurance	Muhammet Emir Dizdar

1.3 Architectural Decision Summary

Bu proje için temel mimari yaklaşım olarak Modular Monolith with Layered Organization seçilmiştir. Sistem tek bir uygulama bütünlüğü içerisinde geliştirilmekte, ancak işlevsel sorumluluklar ayrı modüller halinde yapılandırılmaktadır. Bu tercih, hem geliştirme sürecinde yönetilebilirliği artırmakta hem de modüller arası bağımlılıkların daha kontrollü kurulmasını sağlamaktadır. Ayrıca sistem içerisindeki bazı akışların gevşek bağlı şekilde ilerleyebilmesi amacıyla yapı, gerekli noktalarda event-driven communication yaklaşımıyla desteklenmektedir. Böylece özellikle veri güncelleme, puan hesaplama ve sıralama yenileme gibi süreçlerde modüller arası doğrudan bağımlılık azaltılmakta ve sistem daha esnek bir hale getirilmektedir.

1.4 Scope of This Extension Phase

Bu rapor, projenin ikinci aşamasında geliştirilen Scoring Engine Extension yapısını kapsamaktadır. İlk aşamada geliştirilen takım oluşturma mekanizmasının üzerine, bu aşamada oyuncu performans verilerini işleyen, pozisyona göre puan hesaplayan ve taktik rollerin puanlara etkisini değerlendiren yeni bir puanlama altyapısı eklenmiştir. Bu sayede sistem yalnızca takım kurma işlevi sunan bir yapı olmaktan çıkarılmış, oyun haftası bazlı değerlendirme yapabilen daha kapsamlı bir sisteme dönüştürülmüştür.

1.5 Purpose of the Design Document

Bu dokümanın amacı, geliştirilen uzantının yazılım mimarisi açısından nasıl konumlandığını, sistemde yer alan üst seviye bileşenleri, bu bileşenler arasındaki ilişkileri ve kullanılan tasarım kararlarını açık biçimde ortaya koymaktır. Doküman aynı zamanda UML temsilleri aracılığıyla sistemin yapısal ve davranışsal görünümünü sunarak, sonraki geliştirme fazları için teknik bir temel oluşturmayı hedeflemektedir. Bu kapsamda bileşen yapısı, veri akışı, tasarım desenleri ve temel arayüzler üst seviyede açıklanmaktadır.

2. Introduction & System Overview

2.1 Extension Context: Use Case 2

Following the successful implementation of the Team Builder module in Part 1, the second phase of the project focuses on the development of the **Automated Scoring Engine** as **Use Case 2**. This extension introduces a dedicated scoring subsystem responsible for evaluating player performance data and calculating weekly team scores according to predefined football rules.

The main responsibility of this subsystem is to process player statistics, determine the applicable scoring logic based on player position, and reflect tactical roles such as captaincy in the final point calculation. In this way, the system evolves from a team construction application into a more complete fantasy league platform capable of performance-based evaluation and competitive ranking preparation.

2.2 Key Objectives

The design of the Scoring Engine is guided by three main objectives.

Decoupled Evaluation: The scoring process is separated from team creation and player data representation in order to improve modularity and maintainability. This separation allows the evaluation logic to evolve independently without requiring changes in the core team structure.

Tactical Role Support: The system must support tactical roles such as captain, starter, and bench player by applying score modifiers dynamically. This approach makes it possible to extend player-related behavior without altering the original player models.

Rule Flexibility: Since fantasy football scoring differs by player position, the system is designed to support multiple independent scoring rules. This makes the architecture more extensible and allows position-based calculations to be modified or expanded with minimal impact on the rest of the system.

3. Architectural Design

3.1 Extended Architecture Diagram

The system preserves its **Modular Monolith with Layered Organization** structure while introducing the **Scoring Subsystem** as a core business service. Within this architecture, the user interacts with the system through the **React Interface**, which communicates with the backend via **REST** endpoints exposed by the **FastAPI Controller**. The backend application contains the **Team Builder Service** and the newly added **Scoring Engine Service**, both operating within the same application boundary but with clearly separated responsibilities.

The persistence layer is supported by the **SQLite Database**, while external or periodic football performance data is provided through the **Data Fetcher Scripts**. Inside the scoring subsystem, position-based evaluation is handled by dedicated **Scoring Strategies**, and tactical role effects are applied through **Point Modifiers**. This structure enables the system to keep a clear separation between data management, business logic, and user interaction while integrating the new scoring functionality in a scalable and maintainable way. Figure 1 illustrates the high-level interaction between these major modules and highlights the role of the Scoring Engine as a central business component.

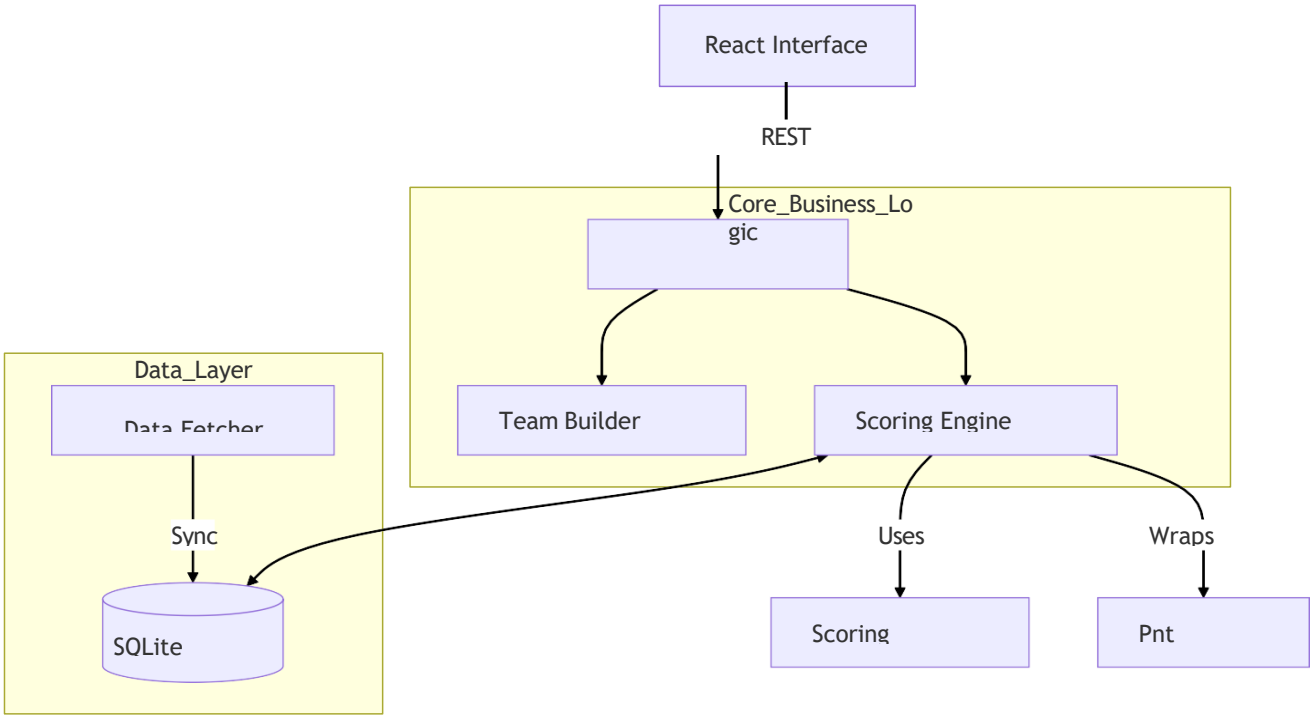


Figure 1: High-Level Module Interaction showing the new Scoring Engine Service.

3.2 Architectural Rationale

The introduction of Use Case 2 required the system to move beyond simple object mapping and adopt a more flexible behavioral design approach. Since player scoring rules differ by position and tactical role, a fixed implementation based on hard-coded conditional structures would reduce maintainability and make future rule changes difficult. For this reason, the architecture was extended with a pattern-oriented design centered on the **Strategy Pattern** and the **Decorator Pattern**.

The **Strategy Pattern** allows the scoring engine to assign a separate scoring algorithm to each player position, such as goalkeeper, defender, midfielder, or forward. This removes the need for a monolithic "if-else" structure and makes the system easier to extend when new rules are introduced. The **Decorator Pattern** is used to apply tactical modifiers dynamically on top of the base scoring logic. In this way, roles such as **captain**, **starter**, or **bench player** can affect the final score without changing the original player models or rewriting the scoring strategies themselves.

As a result, the architectural design improves modularity, extensibility, and maintainability. It also provides a cleaner foundation for future extensions such as leaderboard integration, event-based updates, and additional gameplay mechanics.

.

4. Component Design

4.1 Subsystems and Modules

The scoring subsystem is built around three main components. The **TeamPointCalculator** in calculator.py acts as the main facade of the scoring engine and coordinates the team-wide score calculation process. It iterates over players, selects the appropriate scoring logic, and combines the final results. The **ScoringStrategy** in scoring_strategies.py is an abstract base class that defines the common calculate_score interface for all position-based scoring rules.

The **ModifierStrategy**, also defined in scoring_strategies.py, wraps a base strategy and applies a multiplier to the calculated score in order to support tactical roles such as captain, starter, and bench player.

In addition, concrete strategy classes such as **GoalkeeperStrategy**, **DefenderStrategy**, **MidfielderStrategy**, and **ForwardStrategy** implement the position-specific scoring behavior required by the system.

4.2 Component Dependency Diagram

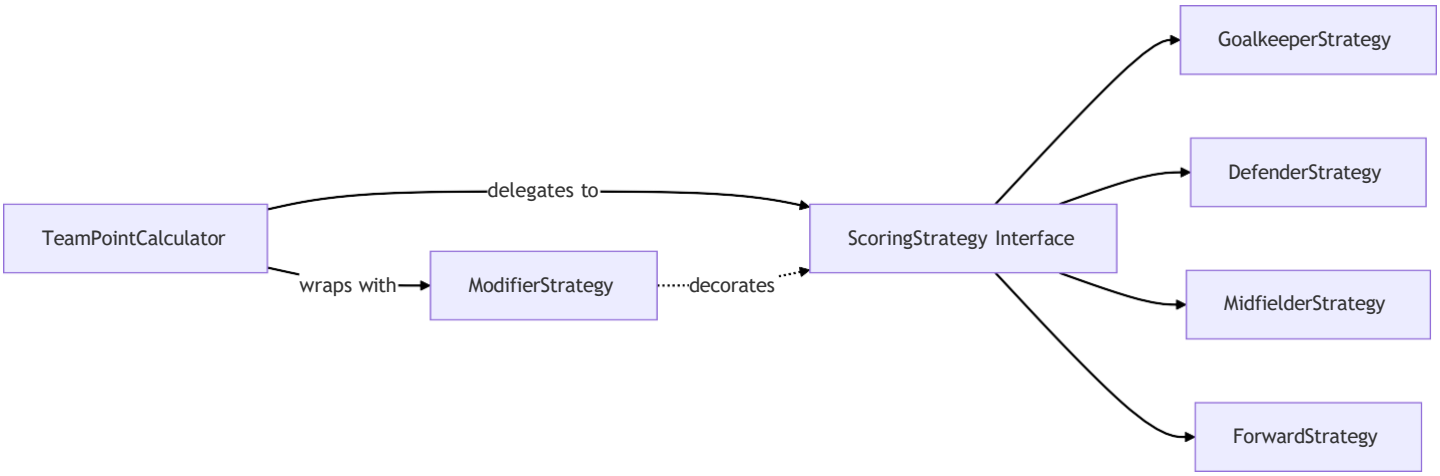


Figure 2: Component Dependency within the Scoring Subsystem.

The dependency structure of the scoring subsystem follows a clean and extensible pattern-based design. The **TeamPointCalculator** depends on the **ScoringStrategy** abstraction rather than directly depending on concrete classes. This allows the calculator to delegate score computation through a common interface. Concrete strategies such as goalkeeper, defender, midfielder, and forward scoring classes implement this interface and provide position-based scoring rules. When a tactical role effect is needed, the selected strategy is wrapped by **ModifierStrategy**, which decorates the base result and applies the required multiplier dynamically. This dependency model reduces coupling and makes the subsystem easier to maintain and extend.

4.3 Component Interfaces

To make the component interactions clearer, the main interfaces of the subsystem are defined at a high level. The **TeamPointCalculator** provides the calculate_team_score(players: List) operation and returns the total team score as an integer. The **ScoringStrategy** interface provides calculate_score(stats: dict) and returns the score of an individual player based on position-specific rules. The **ModifierStrategy** also exposes calculate_score(stats: dict), but internally wraps another strategy together with a multiplier parameter and returns the modified score. At the system level, the scoring subsystem also interacts with external components through REST-based requests and data-fetching processes, while future extensions such as leaderboard refresh and event propagation can be supported through event-based communication. This definition helps clarify both the internal scoring contracts and the subsystem's interaction with the larger application architecture

5. Data Design

5.1 Data Flow Diagram (Scoring)

The scoring data flow begins with raw player performance data, which is received from the API or the stored weekly dataset. This data is passed to the TeamPointCalculator, which identifies the player's position and selects the appropriate concrete scoring strategy. After the base score is calculated according to position-specific rules, the system checks the tactical role of the player, such as captain, starter, or bench status. If necessary, the selected strategy is wrapped by ModifierStrategy, and the final

modified score is produced. At the end of this process, individual player scores are aggregated and returned as the final team score. This flow ensures that raw statistical input is transformed into a structured and extensible scoring result.

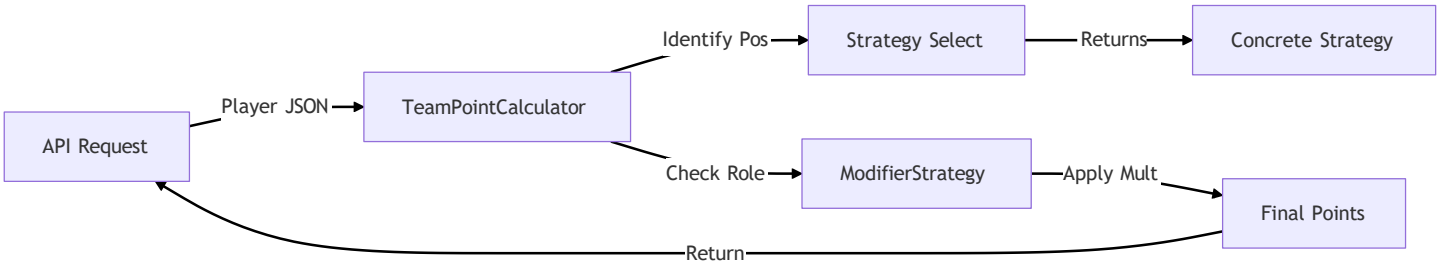


Figure 3: Data Flow from raw player stats to final team points.

6. Design Patterns (Deep Dive)

6.1 Strategy Pattern

The **Strategy Pattern** is used to define a family of scoring algorithms for different player positions. This is necessary because each position has its own scoring rules. For example, goalkeepers may earn points for saves, while midfielders may gain more points for goals or assists. By assigning a separate concrete strategy to each position, the system avoids hard-coded conditional logic and becomes easier to extend and maintain.

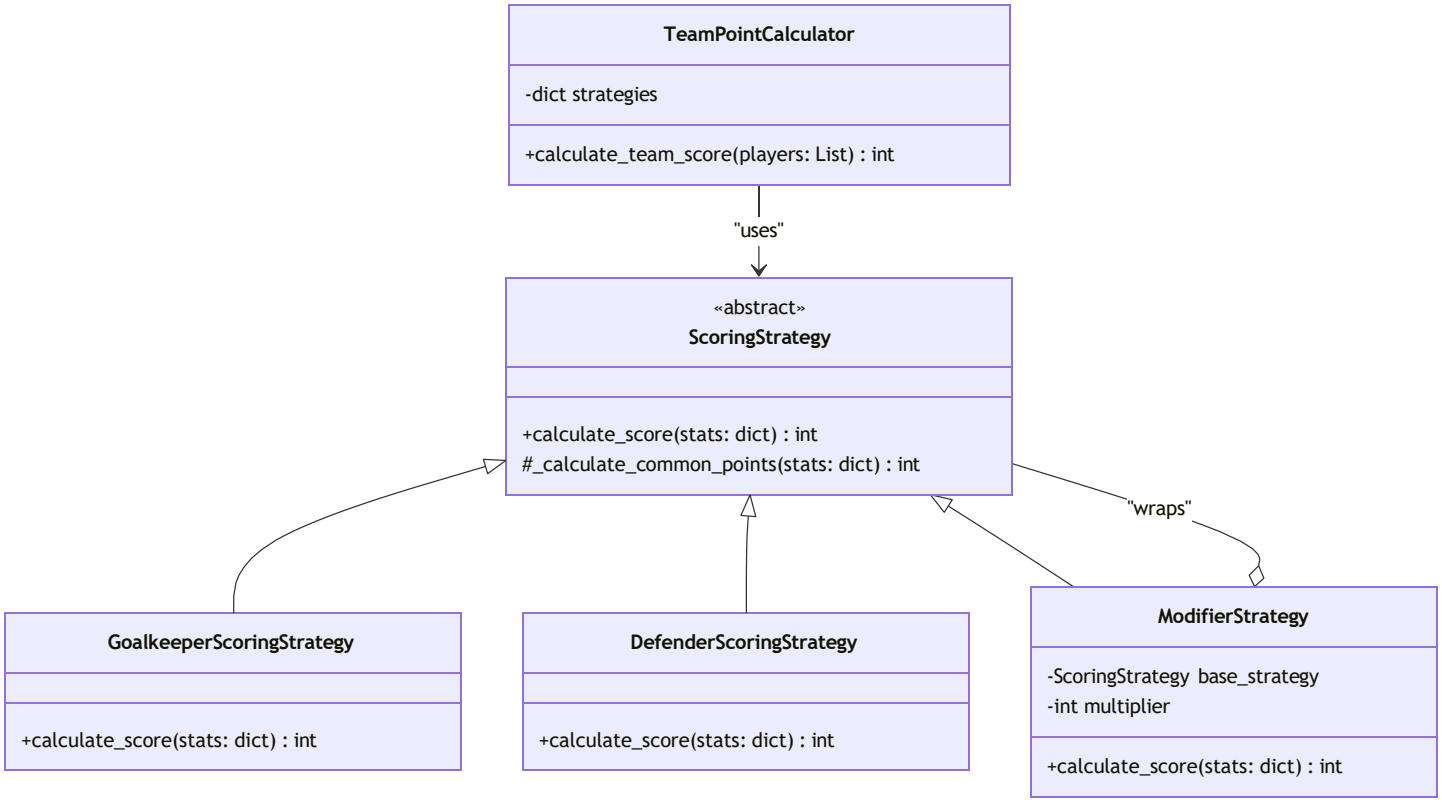
6.2 Decorator Pattern

The **Decorator Pattern** is used to add tactical role effects dynamically on top of the base scoring logic. Instead of changing the original scoring strategy, the system wraps it with **ModifierStrategy** and applies a multiplier according to the player's role. In this design, a **captain** is evaluated with multiplier = 2, a **bench player** with multiplier = 0, and a **starter** with multiplier = 1. This approach keeps the scoring logic modular and allows role-based behavior to be added without modifying the core strategy classes.

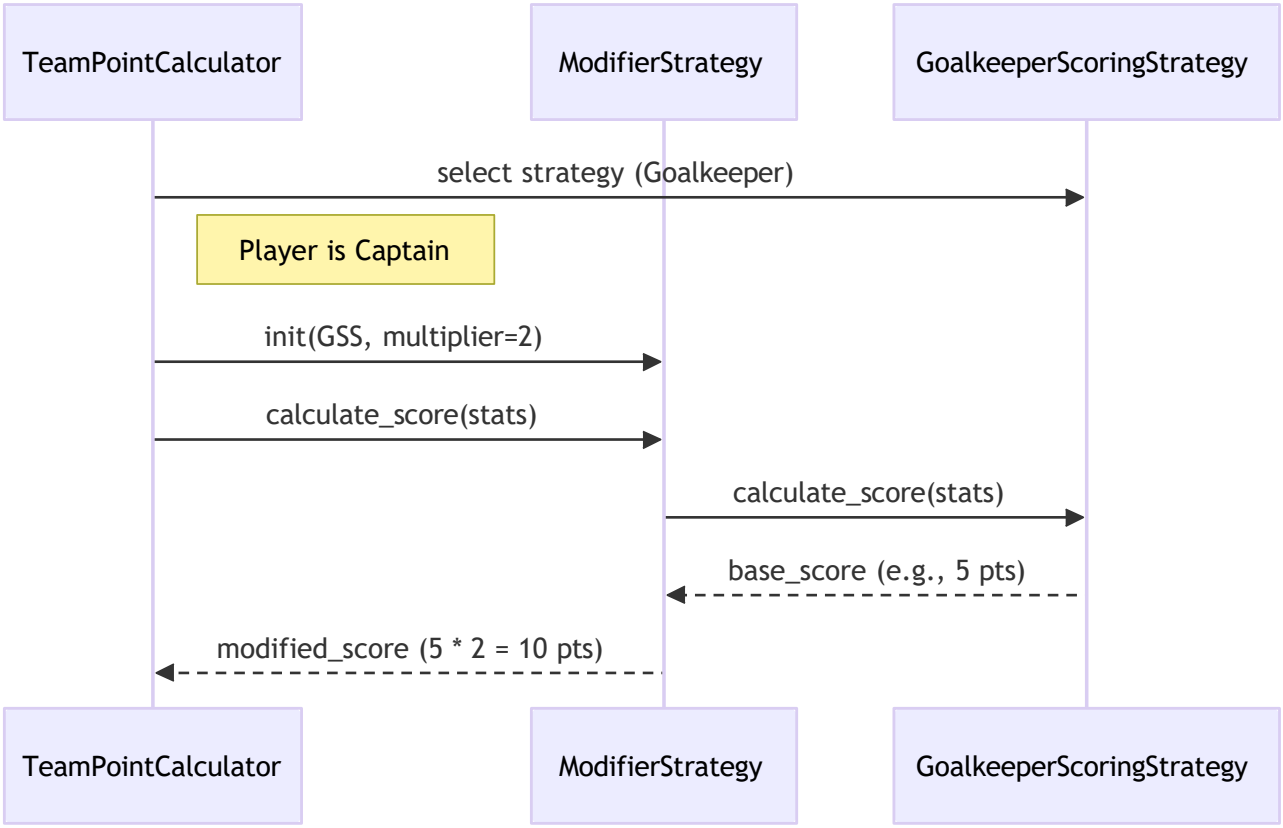
7. Behavioral Representations (Detailed UML)

7.1 Detailed Class Diagram

The detailed class structure of the scoring subsystem is centered on TeamPointCalculator, which stores the available scoring strategies in a dictionary and provides the calculate_team_score(players: List): int operation. This component uses the abstract ScoringStrategy class, which defines the common calculate_score(stats: dict): int interface and the shared helper method _calculate_common_points(stats: dict): int. Concrete scoring classes inherit from this abstraction and implement position-specific scoring behavior, while ModifierStrategy extends the runtime behavior by wrapping a base strategy with a score multiplier.



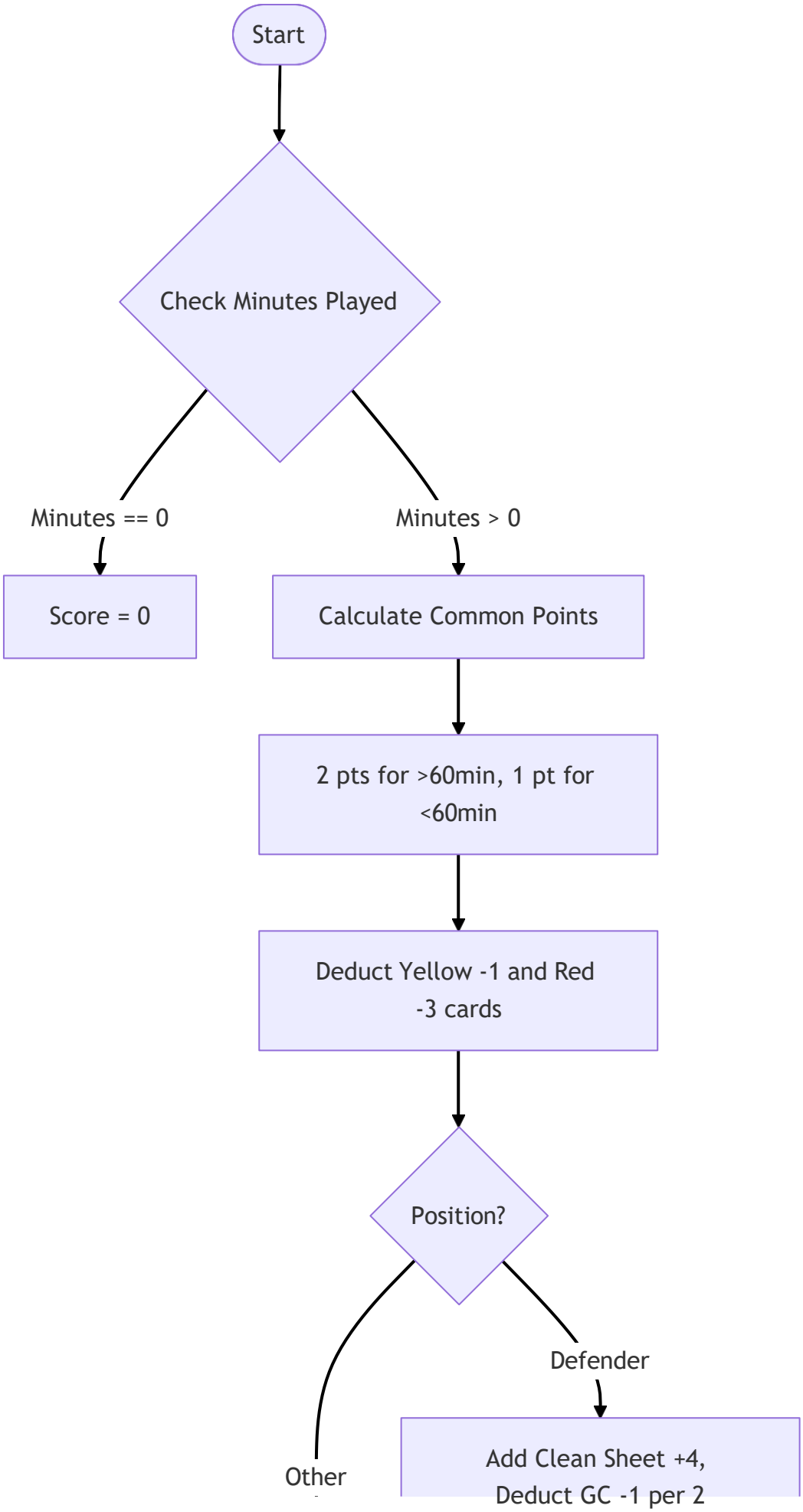
7.2 Sequence Diagram: Player Scoring Lifecycle

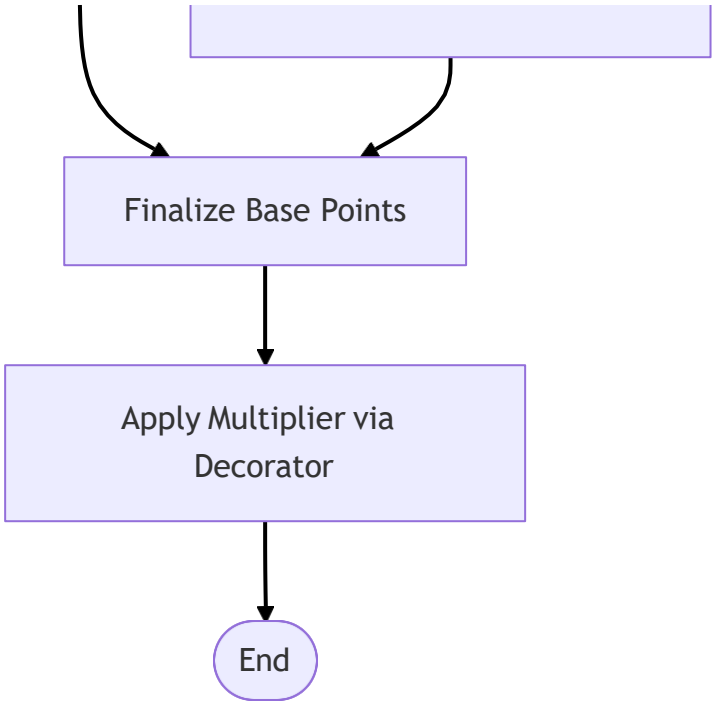


The player scoring lifecycle begins when TeamPointCalculator receives the player list and selects the appropriate scoring strategy according to the player's position. If the player has a tactical role such as captain, the selected strategy is wrapped by ModifierStrategy with the related multiplier. The scoring request is then forwarded to the base strategy, which calculates the player's base score from the input statistics. Finally, the modifier applies its multiplier and returns the updated score to the calculator. This interaction clearly shows how base scoring and tactical role adjustments are combined during runtime.

7.3 Activity Diagram: Internal Scoring Logic

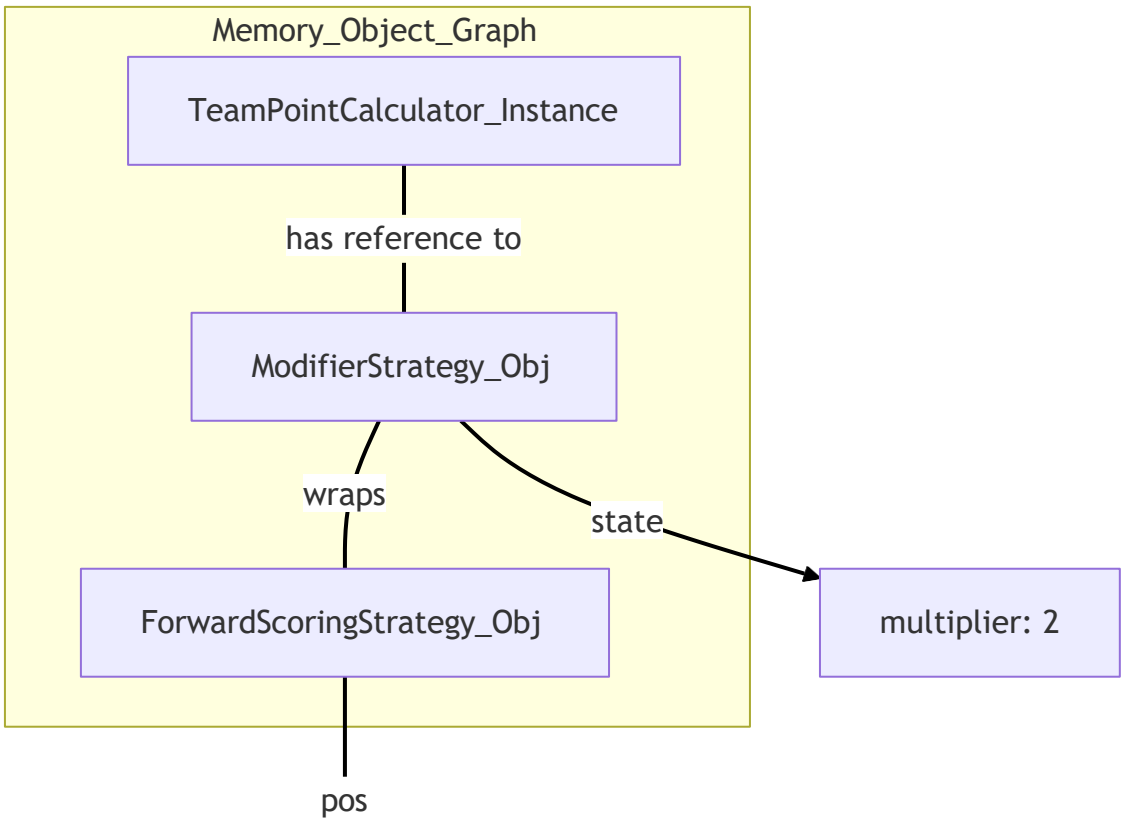
The internal scoring flow starts by checking whether the player has played any minutes. If the number of minutes is zero, the score is set directly to zero. Otherwise, the system calculates common points such as appearance points and disciplinary deductions. After that, the player’s position is evaluated and position-specific scoring rules are applied. For example, defenders may receive clean sheet points and goals-conceded deductions. Once the base score is finalized, the system applies the tactical multiplier through the decorator and produces the final score.

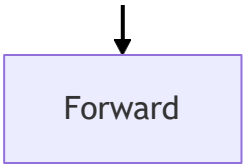

Start
Check Minutes Played
Minutes == 0
Minutes > 0
Score = 0
Calculate Common Points
2 pts for >60min, 1 pt for <60min
Deduct Yellow -1 and Red -3 cards
Position?
Defender
Other
Add Clean Sheet +4, Deduct GC -1 per 2



7.4 Object Diagram: Runtime Decoration

At runtime, the object structure shows TeamPointCalculator holding a reference to a decorated scoring object. For example, when a forward player is selected as captain, the calculator uses a ModifierStrategy object that wraps a ForwardScoringStrategy instance together with a multiplier value of 2. This object-level representation highlights how dynamic decoration is used to extend scoring behavior without changing the underlying strategy implementation.




Forward

8. Performance & Error Handling

8.1 Optimization

To improve efficiency, the available scoring strategies are stored in a dictionary during initialization, allowing the system to perform strategy selection in constant time. In addition, the reuse of shared strategy objects helps reduce unnecessary object creation and prevents excessive memory consumption when many score calculations are performed in sequence.

8.2 Exception Management

The scoring subsystem is designed to handle incomplete or missing external data safely. For this reason, the scoring logic uses .get() operations with default values such as 0 when reading player statistics. This prevents runtime failures caused by missing fields and ensures that the calculation process can continue reliably even when some incoming data is incomplete.

9. Change Log & Future Work

In **v2.0**, the project was extended with the full implementation of **Use Case 2**, introducing the automated scoring engine together with the **Strategy** and **Decorator** patterns. As the next phase of the project, **Use Case 3** will focus on **Global Leaderboard and Competitive Ranking** functionality. This planned extension will introduce a leaderboard structure that keeps updated rankings across teams and refreshes standings whenever scores change. It is expected to rely on additional design support such as **Singleton** for centralized ranking management and **Observer**-style updates for reflecting score changes in the ranking view.